GAME ASSET PIPELINE

ARCHITECTURE SPECIFICATION

A Reusable Framework for AI-Generated 2D Isometric Browser Games

Version 1.0 — March 2026

First Implementation: Shattered Veil

Cameron Chojnacki — Creative Director & Technical Architect

1. System Overview

This document defines a reusable, game-agnostic architecture for producing 2D isometric sprite-based browser games using AI-generated art. The pipeline is designed to be human-led and AI-driven: creative decisions (art direction, style selection, quality assurance, game design) remain with the creative director, while mechanical execution (asset generation, background removal, batch processing, file management) is automated through AI tooling.

The architecture separates into three tiers that change at different rates:

Tier	What It Contains	Changes When
Tier 1: Pipeline	ComfyUI workflows, orchestration scripts, post-processing tools, Phaser rendering engine	Rarely — only when switching base AI models or rendering engines
Tier 2: Style Layer	LoRA checkpoints, art direction prompts, trigger words, style block configuration	Per game project — each game has its own trained visual style
Tier 3: Content	Asset manifest entries, variant descriptions, class presets, equipment definitions	Continuously — every new piece of equipment, character, or environment

Human-in-the-Loop Touchpoints

These are the decision points where the creative director reviews and approves:

- LoRA epoch selection (which training checkpoint captures the desired style)
- Art direction decisions (dark/gritty vs bright/stylized, proportions, color palette)
- Generated asset QA (reviewing outputs, accepting or requesting regeneration)
- Class preset composition review (does this character look right with this equipment?)
- Animation keyframe approval (do the casting poses feel right?)
- Cross-LoRA style consistency checks (do weapons match characters match environments?)

2. Sprite Layer System

Every character in the game is rendered as an ordered stack of transparent PNG layers composited at runtime by the browser engine. The layer types are universal across all game genres. The contents are game-specific.

2.1 Layer Definitions

Layer	Name	Nullable	Gen Method	Description
0	base_body	No	Isolated	Full clothed character base. Defines class silhouette. Hands in open grip pose.

1	back_item	Yes	Isolated	Capes, cloaks, backpacks, quivers, wings. Renders behind body in most directions.
2	headwear	Yes	Isolated	Helmets, hoods, circlets, hats. May hide hair layer via interaction flag.
3	hand_main	Yes	Isolated	Primary weapon or tool. Includes handle overlap to cover grip hand.
4	hand_off	Yes	Isolated	Shield, tome, focus item, off-hand weapon.
5	vfx	Yes	Isolated	Spell effects, auras, elemental particles. May be animated sprite sequences.

The base_body layer is NOT a bare anatomical figure. It is a fully clothed character in their class outfit. This is a deliberate design decision based on validation testing: the LoRA produces consistent chibi-proportioned results when generating clothed characters, but falls back to semi-realistic proportions for unclothed bodies. Visual variety comes from swapping weapons, headwear, back items, and VFX layers on top of different class base bodies.

2.2 Simplified Layer Architecture

Based on end-to-end validation testing, the architecture uses a simplified 6-layer stack rather than the more granular 13-layer system initially considered. This reduces the number of assets to generate, keeps all generation within the LoRA's training distribution, and simplifies engine compositing. The tradeoff is that torso/belt/legs/feet are not individually swappable — they change at the class level. This is appropriate for the target game genre where class silhouette matters more than individual clothing pieces.

Sticker Border Handling

Each generated asset has a white sticker-like highlight border (2–3px). When layers overlap, these borders stack visually.

Options: (A) Generate equipment layers without sticker border, only body gets it.

(B) Accept the overlap as a stylistic paper-doll aesthetic.

Recommendation: Option B for initial release — it reinforces the stylized look. Revisit if player feedback indicates it looks unpolished.

3. Directional Sprite System

Isometric 2.5D rendering requires 8 facing directions. To minimize generation work, 5 unique directions are generated and 3 are produced by horizontal mirroring.

3.1 Direction Map

Direction	Code	Facing	Source

South	S	Toward camera (front view)	Generated
South-East	SE	Front-right diagonal	Generated
East	E	Right profile	Generated
North-East	NE	Back-right diagonal	Generated
North	N	Away from camera (back view)	Generated
South-West	SW	Front-left diagonal	Mirror of SE
West	W	Left profile	Mirror of E
North-West	NW	Back-left diagonal	Mirror of NE

This means every layer asset exists as 5 PNG files. A single weapon with 5 directions = 5 generation API calls. A fully-equipped class across all layers and directions = approximately 25–30 generations.

3.2 Direction-Dependent Render Order

The layer render order is NOT fixed. It changes per direction because items that are in front of the character when facing south appear behind the character when facing north. The engine reads a render order lookup table:

Direction	back_item	base_body	headwear	hand_off	hand_main	vfx
S (front)	Behind (–1)	Base (0)	Above (1)	Above (3)	Above (4)	Top (5)
SE	Behind (–1)	Base (0)	Above (1)	Behind (–1)	Above (2)	Top (3)
E	Behind (–1)	Base (0)	Above (1)	Behind (–1)	Above (2)	Top (3)
NE	Above (1)	Base (0)	Above (2)	Behind (–1)	Behind (–1)	Top (3)
N (back)	Above (1)	Base (0)	Above (2)	Behind (–1)	Behind (–2)	Top (3)

Negative render order values indicate layers rendered behind the base body. The engine sorts all layers by render order before drawing. Mirrored directions (SW, W, NW) use the same render order as their source direction (SE, E, NE).

LoRA Training Direction Bias

The character LoRA was trained primarily on three-quarter view images (approximately SE direction).

Expect strongest generation quality at S and SE, weakest at N (back view).

Back-view assets may require additional prompt engineering, cherry-picking across seeds, or supplemental training images.

4. Hand and Weapon System

The base body is generated with hands in an open grip pose — fingers slightly curled as if holding a weapon handle, but holding nothing. Weapon assets include a small overlap region that covers the grip area of the hand. The weapon handle/pommel paints over the empty grip.

4.1 Anchor Point System

Each base body defines anchor coordinates for attachment points. These are stored as pixel ratios relative to the 1024×1024 canvas and calibrated once per body type per direction during initial asset QA.

Anchor	Purpose	Typical Position (S direction)
hand_main	Primary weapon attachment	Right side, mid-torso height (~75% X, ~55% Y)
hand_off	Off-hand item attachment	Left side, mid-torso height (~25% X, ~55% Y)
head_top	Headwear placement	Center, top of head (~50% X, ~15% Y)
back_center	Cape/backpack attachment	Center, behind shoulders (~50% X, ~35% Y)

Anchor coordinates shift per direction. When the character faces East, the main hand anchor moves toward the right edge of the canvas. When facing North, it moves behind the body. The manifest stores a complete anchor map: body_type × direction × anchor_name → (x, y) coordinates.

4.2 Weapon Design Constraints

- Weapons must include handle/grip region that overlaps and covers the hand anchor area
- Weapon scale must be consistent relative to the body across all 5 directions
- Weapons are generated at the same 1024×1024 canvas size as bodies for spatial consistency
- Two-handed weapons (staves, bows) span both hand anchors and may require custom anchor overrides

5. Animation System

The animation system prioritizes engine-driven tweens on static sprites over pre-rendered frame sequences. Approximately 70–80% of all animation is achieved through programmatic transforms (position, rotation, scale, opacity) applied to static layer sprites. The remaining 20–30% requires generated keyframes or sprite sequences for effects that involve actual pixel content changes.

5.1 Tween-Only Animations (No Generated Frames)

State	Technique	Description

Idle	Y-axis sine wave, 2–3px amplitude, 2s cycle	Subtle breathing bob on entire layer stack
Walk	Y-axis bob 3–4px, faster cycle + X/Y translation	Bouncy movement along path. No leg animation — bob sells motion at chibi scale
Hit / Damage	8px offset opposite to hit direction + white flash overlay	Knockback snap with 100ms recovery
Death	15° rotation toward ground + opacity fade to 0	Tilt and dissolve over 500ms
Buff / Debuff	VFX layer opacity pulse or tint overlay	Glow effect on character
Equip Swap	100ms fade out old layer, 100ms fade in new	Smooth equipment transition
Melee Attack	Weapon layer rotates 60–90° arc around hand anchor, 200–300ms	Sword swing. Body leans slightly into swing direction
Block / Parry	Off-hand layer translates up + rotates to cover, 150ms	Shield raise

5.2 Animations Requiring Generated Keyframes

State	Frames Needed	Why Tweens Are Insufficient
Spell Cast	2–3 body keyframes per direction	Arm position changes fundamentally: arms at sides → arms raised → hands extended. Cannot fake with translation.
Bow Draw	2 body keyframes per direction	Arm silhouette changes: bow held forward → bow drawn back.
Channel	1 alternate body keyframe per direction	Sustained alternate pose with arms in channeling position.

Frame budget for body keyframes: 3 poses × 5 directions × N body types. For 4 body types, this is 60 additional body generations. Each takes ~15–20 seconds on an RTX 4080.

5.3 VFX Sprite Sequences

Effect	Frames	Approach
Fireball projectile	4–6 looped	Generate 2–3 keyframes with LoRA, RIFE interpolate to target count
Ice burst	6–8 one-shot	Generate 3 keyframes, interpolate

Heal pulse	4–6 looped	May be achievable with pure tween (opacity + scale pulse on glow sprite)
Lightning strike	3–4 one-shot	Generate each frame individually — lightning is too erratic for interpolation
Impact / hit spark	3–4 one-shot	Small, fast effect. Reusable across skills

Total VFX frame budget: approximately 30–50 generated frames across all effect types. These are generated using the VFX LoRA (or base Flux with detailed prompting as placeholder) and optionally expanded via RIFE frame interpolation.

5.4 What This System Does NOT Support

- Fluid hand-drawn animation (Hollow Knight / Cuphead style) — those require frame-by-frame artist work
- 3D rotation of equipment — items face a fixed direction per sprite direction
- Physics-based cloth or hair simulation — capes are static per direction with optional tween sway
- Detailed facial animation — the head is one sprite per expression, no lip sync or eye tracking
- Complex creature animation with many moving parts — tentacles, multi-joint wings require dedicated sprite sheets

6. LoRA Training Strategy

The visual style is enforced through Flux.1 Dev LoRA fine-tuning. Multiple specialized LoRAs are trained per game project, each focused on a specific asset category. This produces tighter style consistency within each category than a single general-purpose LoRA.

6.1 LoRA Registry (Shattered Veil)

LoRA	Status	Dataset Size	Purpose	Training Priority
Characters	Trained (epoch 8)	34 images	Base bodies, heads, facial features, hair styles	Complete
Equipment & Armor	Pending	30–40 images	Chest armor, robes, belts, helmets, boots, capes	Priority 1
Weapons & Items	Pending	25–35 images	Swords, staves, shields, orbs, daggers, bows, potions	Priority 2

Environments	Pending	30–40 images	Dungeon tiles, walls, doors, props, parallax backgrounds	Priority 3
VFX & Spells	Pending	25–30 images	Fire, ice, lightning, healing, buff/debuff effects	Priority 4
UI & Icons	Evaluate	20–30 images	Skill icons, inventory frames, UI elements	May not need LoRA

6.2 Bootstrap Strategy

Training the Equipment and Weapons LoRAs requires reference images in the Shattered Veil style, but those LoRAs do not exist yet. The solution: use the already-trained Characters LoRA to generate full characters wearing target equipment, then isolate the equipment pieces using SAM segmentation (ComfyUI Impact Pack) or manual cropping. These isolated pieces become the training dataset for the specialized LoRAs.

Cross-LoRA Style Consistency

Risk: Separately trained LoRAs develop subtle style drift (line weight, color saturation, shadow hardness).

Mitigation: Strict style anchors in every prompt (outline weight, flat fills, sticker border).

QA checkpoint: After training each new LoRA, generate test assets and composite them with existing LoRA outputs. Review for visual cohesion before proceeding to batch generation.

This is a human review point — Cameron evaluates cross-LoRA composites.

7. Generation Pipeline

7.1 ComfyUI Workflows

Two reusable ComfyUI workflow templates handle all asset generation. Both are exported as API-format JSON for programmatic submission.

Workflow	Use Case	Node Chain
Isolated Part	Weapons, headwear, VFX, back items	Load Flux → Load LoRA → DualCLIPLoader → CLIP Encode → KSampler → VAE Decode → Inspyrenet Rembg → Save Image
Body-Relative	Equipment on body → segment → extract	Same generation chain → SAM Segment target region → Inspyrenet Rembg → Save Image

7.2 Standard Generation Parameters

Parameter	Value	Notes
Base Model	Flux.1 Dev (flux1-dev.safetensors)	Full precision diffusion model
CLIP	DualCLIPLoader: clip_l + t5xxl_fp16, type=flux	Both text encoders required for Flux
VAE	ae.safetensors	Flux autoencoder
Canvas Size	1024 × 1024	Standard for all assets
Sampler	euler / simple scheduler	Consistent across all generations
Steps	20	Sufficient for LoRA-guided generation
CFG	1.0	Flux does not use traditional classifier-free guidance
LoRA Strength	0.85 (model and clip)	Adjustable per LoRA based on testing
Background Removal	Inspyrenet Rembg	Free, handles cel-shaded edges cleanly

7.3 Prompt Architecture

Prompts are generated programmatically from the asset manifest. The orchestration script combines style block fields with per-variant descriptions:

{trigger_word}, {variant_description}, {direction_instruction}, {global_art_direction}, {background}, {negative_anchors}

This ensures every generated asset shares consistent style tokens. The creative director writes only the variant_description field. Everything else is templated from the manifest.

8. Browser Rendering Architecture

8.1 Technology Stack

- Phaser 3 — 2D game engine with WebGL rendering, sprite management, tween system, input handling
- Colyseus — multiplayer server framework, authoritative state management, room-based architecture
- WebSocket — real-time client-server communication for player state sync

8.2 Sprite Compositing at Runtime

The engine reads the asset manifest and composites character layers at runtime. For each visible character:

- **1.** Read the character's current direction from game state
- **2.** Look up render order for that direction from the manifest
- **3.** Sort all equipped layers by render order
- **4.** For each layer, load the correct directional sprite (e.g., sword_SE.png)
- **5.** Position each layer at its anchor point relative to the base body
- **6.** If direction is SW, W, or NW: use the SE, E, or NE sprite with horizontal flip
- **7.** Apply any active tween animations to the layer stack

8.3 Performance Budget

Compositing 6 transparent PNG layers per character at 60fps is trivial for WebGL. Phaser uses GPU-accelerated sprite batching. Conservative estimates:

Metric	Budget	Notes
Characters on screen	50+	Each with 6 composited layers
Draw calls	~300 per frame	Phaser batches same-texture sprites
Texture memory	~200MB loaded	Sprite atlases, lazy-loaded per area
Target framerate	60fps	Achievable on any modern browser with WebGL
Mobile target	30fps minimum	Reduced draw distance, simplified VFX

8.4 Asset Loading Strategy

With 12 classes × 6 layers × 5 directions × 3 animation states, the total asset count approaches 1,000+ individual PNGs. Direct loading is impractical. The engine uses:

- **Sprite Atlasing:** Multiple sprites packed into single texture sheets (e.g., all sword directions in one atlas). Reduces HTTP requests and GPU texture swaps.
- **Lazy Loading:** Only the current player's class + nearby visible enemies are loaded. Remote players load on proximity. Area transitions preload the next zone's assets.
- **Asset Manifest Indexing:** The manifest acts as a lookup table. The engine never scans directories — it reads the manifest to know exactly which files to load for a given class/equipment combination.

9. Asset Manifest Specification

Each game project has one master manifest JSON file that defines all assets, their relationships, and generation parameters. The manifest serves three roles:

- **Generation source of truth:** The orchestration script reads the manifest to know what to generate and how.
- **Engine runtime config:** Phaser reads the manifest to know how to composite and render characters.
- **Content management:** Adding new content = adding entries to the manifest. No code changes required.

The manifest schema includes: project metadata, style block (LoRA references, art direction, trigger word), layer definitions (render order, anchor points, generation method), variant entries (per-layer equipment/item variants with descriptions and tags), class presets (pre-defined layer combinations per character class), and pipeline configuration (ComfyUI host, workflow references, post-processing settings).

A complete reference manifest for Shattered Veil (shattered_veil_manifest.json) has been created and validated alongside this document.

10. Project Plan

The plan is structured around the principle that Cameron drives creative vision and quality decisions while AI handles mechanical execution. Each phase identifies who does what.

Phase 1: Game Design (Cameron Solo)

Duration: 2–4 weeks. No AI tooling needed.

- **Story & World:** Define the setting, lore, tone, factions, conflict. What makes this world worth exploring?
- **Core Loop:** What does a player do in a typical 30-minute session? What's the moment-to-moment gameplay?
- **Class System:** How many classes at launch? What makes each feel distinct? How does the skill-mixing system work?
- **Progression:** What do players work toward? Loot? Story? Ranks? Seasonal content?
- **Visual Identity:** Dark and gritty, or bright and stylized? What mood does the art set? Reference images/games that capture the vibe.
- **Scope Lock:** What ships in v0.1? A single dungeon, 3 classes, basic loot? Define the vertical slice.

Deliverable from Phase 1

A Game Design Document (GDD) covering story, systems, class definitions, progression, and visual direction.

This does not need to be long. 5–10 pages of clear decisions is worth more than 50 pages of exploration.

Cameron can write this in any format. It will be used as the input for all subsequent AI-driven work.

Phase 2: Art Direction & LoRA Training (Cameron + AI)

Duration: 2–3 weeks.

Task	Who	Details
Gather reference art for each asset category	Cameron	Find 5–10 images per category that represent the target look
Prepare training datasets (crop, clean, caption)	AI (orchestration script)	Automated captioning with manual review
Review and adjust captions/descriptions	Cameron	Ensure captions accurately describe what the LoRA should learn
Train Equipment LoRA	AI (FluxGym)	Automated process, ~2 hours on RTX 4080
Select best epoch for Equipment LoRA	Cameron	Same evaluation process used for Characters LoRA
Train Weapons LoRA	AI (FluxGym)	Same process
Select best epoch for Weapons LoRA	Cameron	Evaluate batches, pick the checkpoint that feels right
Cross-LoRA composite QA	Cameron	Generate test composites, verify visual cohesion across LoRAs
Train Environment LoRA (if needed for v0.1)	AI + Cameron	May defer to Phase 4 if v0.1 scope is small

Phase 3: Asset Generation & QA (Cameron + AI)

Duration: 2–3 weeks.

Task	Who	Details
Build class presets in manifest	Cameron	Define which equipment combos make each class look right
Batch-generate all base bodies (per class, per direction)	AI (orchestration script)	Automated ComfyUI API submission
QA review base bodies	Cameron	Flag any that need regeneration, adjust prompts

Batch-generate weapons (per type, per direction)	AI	Automated
QA review weapons	Cameron	Check style consistency, scale relative to bodies
Generate headwear, back items, VFX	AI	Automated per manifest
QA review all equipment	Cameron	Full composite review using browser compositor tool
Generate casting keyframes (2–3 poses per body)	AI	Per direction, per body type
QA review animations	Cameron	Preview in Phaser prototype, approve or adjust
Anchor point calibration	Cameron + AI	Open each base body, mark hand/head positions
Build sprite atlases	AI (TexturePacker or script)	Pack sprites into optimized sheets

Phase 4: Engine Development (Dev Team + AI)

Duration: 4–6 weeks.

Task	Who	Details
Scaffold Phaser 3 project	Dev team / Claude Code	Project structure, build tooling, dev server
Implement manifest-driven sprite compositor	Dev team	Layer loading, render order by direction, anchor positioning
Implement tween animation system	Dev team	Idle, walk, attack, hit, death states
Implement keyframe animation for casting	Dev team	Frame stepping through generated body keyframes
Build procedural dungeon generator (BSP)	Dev team	Isometric tile-based dungeon with room/corridor structure
Integrate Colyseus multiplayer	Dev team	Room management, state sync, player interpolation
Implement class/skill system from GDD	Dev team	Based on Cameron's game design document
UI implementation	Dev team	HUD, inventory, skill bar, minimap
Mobile responsiveness pass	Dev team	Touch controls, viewport scaling, performance tuning

Phase 5: Integration & Polish (All)

Duration: 2–4 weeks.

- **Asset integration testing:** All generated assets loaded in engine, full composite review in-game.
- **Animation tuning:** Adjust tween parameters, timing, easing curves for feel.
- **Environment art pass:** Generate and integrate dungeon tiles, props, backgrounds.
- **VFX polish:** Generate spell effect sprite sequences, integrate with skill system.
- **Performance profiling:** Browser testing across Chrome, Firefox, Safari, mobile.
- **Playtesting:** Cameron plays through, identifies what feels wrong, what needs adjustment.

Total Timeline Estimate

Phase 1: 2–4 weeks (Cameron solo)

Phase 2: 2–3 weeks (Cameron + AI)

Phase 3: 2–3 weeks (Cameron + AI)

Phase 4: 4–6 weeks (Dev team)

Phase 5: 2–4 weeks (All)

Total: 12–20 weeks to playable prototype with multiplayer

Note: Phases 2–3 can overlap with Phase 4 if dev team starts engine work while assets are being generated.

11. Dev Team Handoff Package

When Cameron is ready to bring on a development team, the handoff package contains everything they need to start building without ambiguity:

- **This architecture document —** defines the rendering system, layer stack, animation approach, and technical constraints.
- **Game Design Document —** defines what the game IS (story, systems, progression, classes). Written by Cameron in Phase 1.
- **Asset manifest (JSON) —** complete definition of all assets, variants, anchor points, render orders.
- **ComfyUI workflow templates (JSON) —** reusable generation workflows for isolated parts and body-relative generation.
- **Orchestration script (Python) —** reads manifest, submits generation jobs to ComfyUI, runs post-processing.
- **Generated asset library —** all sprite PNGs organized by layer/direction, plus sprite atlases.
- **Browser compositor tool (HTML) —** for previewing layer composites during development.
- **LoRA checkpoints —** all trained model files, with documentation of which epoch was selected and why.

The development team does not need to understand LoRA training, ComfyUI, or the generation pipeline. They receive finished PNG assets and a manifest that tells them how to composite and animate those assets in Phaser. The pipeline remains Cameron's domain for ongoing content creation.

12. Reusability for Future Games

This architecture is designed to be reused across multiple game projects. Starting a new game requires:

- **New LoRA training:** Train style LoRAs on reference art for the new game's visual identity. ~2 hours per LoRA on RTX 4080.
- **New asset manifest:** Duplicate the schema template, fill in game-specific layer variants, equipment, classes. Layer types remain the same.
- **New art direction block:** Update trigger word, style description, color palette in the manifest style block.

What does NOT change between games: ComfyUI workflow templates, orchestration script, post-processing pipeline, Phaser sprite compositor engine, layer type definitions, render order system, animation tween library, anchor point system.

A sci-fi game, a baseball RPG, and a fantasy dungeon crawler all use the same pipeline. The art is different. The architecture is identical.

End of Architecture Specification